Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Carcassonne

Cahier des charges

29 décembre 2025
Version 1.0

Ce projet a pour objectif de développer un logiciel permettant de jouer à Carcassone (règles de 2012), le jeu de base uniquement (on ne considèrera que l'extension '*The River*'). Le programme doit permettre à deux à huit joueurs de s'affronter en respectant les règles du jeu.

Mots-clefs: Jeu de tuiles, deux à huit joueurs, stochastique, information parfaite.

1 Généralités

1.1 Langage de programmation

Le langage de programmation doit être choisi parmi :

- **C** : C11 (ISO/IEC 9899 :2011).
- **Java** : Java SE 21+.
- **Python** : Python 3.10+.

1.2 Style de codage

Le *coding style* du programme doit respecter :

- **C** : le *coding style* de GNU.
- **Java** : le *coding style* de Google.
- **Python** : le *coding style* du PEP8.

1.3 Documentation

Le programme et le code source doit être documenté : manuel utilisateur, option d'aide en ligne de commande et commentaires dans le code source. En outre, on utilisera :

- **C** : Doxygen.
- **Java** : Javadoc.
- **Python** : Sphinx.

1.4 Langue par défaut dans le code

La documentation, le nom des variables, des fonctions et des fichiers doivent être en anglais.

1.5 Système cible

Le programme doit fonctionner sur des systèmes GNU/Linux.

1.6 Tests

Le programme doit être testé et validé par suffisamment de tests pour garantir son bon fonctionnement (couverture globale d'au moins 85%). Les tests doivent être automatisés et facile à lancer via le build-system.

1.7 Bugs

Aucun bug menant à un crash ne doit être présent dans le programme. Les spécifications doivent être respectées et les erreurs doivent générer des messages d'erreur explicites.

1.8 Performances

Les performances du programme doivent être mesurées et documentées pour différentes entrées du programme ainsi que pour ses cas limites.

1.9 Bibliothèques

Seule la bibliothèque standard du langage choisi et celles qui sont listées ici sont acceptées.

1.10 Build-system

Le '*build-system*' utilise les outils suivants :

- **C** : `CMake`.
- **Java** : `Maven`.
- **Python** : `pyproject.toml` + `setuptools`.

1.11 Frameworks de tests

Le *framework* de test est basé sur :

- **C** : `CTest` + `googletest` + `gcovr`.
- **Java** : `JUnit` + `Jacoco`.
- **Python** : `pytest` + `coverage`.

1.12 Gestion des options

Les options en ligne de commande sont gérées par :

- **C** : le module `getopt_long`.
- **Java** : la bibliothèque `commons-cli`.
- **Python** : le module `argparse`.

1.13 Bibliothèque graphique

La bibliothèque graphique utilisée est :

- **C** : LibGtk 4.x.
- **Java** : JavaFx.
- **Python** : PyGObject.

1.14 Internationalisation

Le *framework* d'internationalisation est basé sur :

- **C** : la bibliothèque `gettext`.
- **Java** : `ResourcesBundle` et `Locale`.
- **Python** : le module Python `gettext`.

1.15 Apprentissage automatique

L'apprentissage automatique doit être basé sur:

- **C**: la bibliothèque `tensorflow` (C API).
- **Java**: la bibliothèque `tensorflow` (Java API).
- **Python**: la bibliothèque `keras`.

2 Besoins généraux

F1. Lancement du programme

Le programme doit se lancer depuis la ligne de commande via un exécutable ou un script du nom du projet.

```
#> carcassonne [OPTIONS] [ARGUMENTS]
```

Il doit accepter à minima les options suivantes :

- '-h' ou '--help' : affiche l'aide du programme.
- '-V' ou '--version' : affiche la version du programme.
- '-v' ou '--verbose' : accroît la verbosité du programme.
- '-d' ou '--debug' : affiche les messages de debug.

Si une option est incorrecte, le programme doit afficher un message d'erreur et l'aide du programme.

Les messages destinés à l'utilisateur seront écrits sur la sortie standard (`stdout`) et les messages d'erreurs devront être écrits sur la sortie d'erreur standard (`stderr`). De plus, le code de retour du programme devra être 0 en cas de succès et 1 (ou un autre code d'erreur) en cas d'erreur.

F2. Fichier de configuration

Les valeurs par défaut des options du programme doivent pouvoir être redéfinies via un fichier de configuration au format `INI`. Ce fichier de configuration doit être nommé '`.carcassonnerc`' et être placé à la racine du répertoire de l'utilisateur (`HOME`) qui lance le programme. Le fichier de configuration doit être lu au démarrage du programme s'il est présent et les options qu'il contient doivent être utilisées par défaut. Les options passées en ligne de commande doivent supplanter les options du fichier de configuration.

Si le fichier de configuration n'est pas présent, le programme doit créer un fichier de configuration minimal et se lancer normalement en respectant les options par défaut. Si le fichier de configuration est présent mais n'est pas valide, le programme doit afficher un message d'avertissement sur la console et se lancer normalement sans écraser le fichier de configuration. Ci-dessous un exemple de fichier de configuration valide (le nom des options sont là à titre d'exemple) :

```
[defaults]
verbose = true
blitz = false
timeout = 30
```

F3. Internationalisation (`i18n`)

La langue par défaut du logiciel sera l'anglais. Le français, au moins, doit aussi être supporté. Si les variables d'environnement '`LANG`' ou '`LC_ALL`' sont définies, le programme doit les utiliser pour déterminer la langue à utiliser. Si la langue n'est pas supportée, le programme doit afficher un message d'avertissement et se lancer en anglais (la langue par défaut).

3 Options au lancement

F4. Interface en ligne de commande

Par défaut, le programme démarre une interface en ligne de commande sur une nouvelle partie avec les options par défaut si aucune option spécifique n'est donnée. Si des options sont données, celles-ci seront appliquées à la partie qui est lancée. Enfin, si on met en argument un fichier de sauvegarde, le programme doit lancer une partie à partir de ce fichier.

F5. Interface graphique

Le programme doit posséder une interface graphique qui permette les mêmes interactions que sur le CLI avec l'utilisateur via une fenêtre graphique. Cette interface graphique sera lancée si l'option '`-g`', '`--gui`' est présente au lancement.

F6. Mode `blitz`

L'option '`-b`', '`--blitz`' lance une partie en mode blitz. Dans ce mode, un temps maximum est donné à chaque joueur pour la partie. Dès que le temps d'un des joueurs est écoulé, il perd la partie et doit en être notifié.

De plus, '`-t TIME`', '`--time TIME`' spécifie la limite de temps en minutes pour chaque joueur en mode blitz. Par défaut, le temps de réflexion est de 30 minutes. Si cette option est donnée sans l'option '`-b`', '`--blitz`', elle sera ignorée mais un message d'avertissement sera affiché.

F7. Mode `contest`

L'option '`-c`', '`--contest`' lance le programme en mode '`contest`'. Dans ce mode, le programme doit lire un fichier donné en argument qui contient une position du jeu et répondre sur la sortie standard quel coup jouer.

F8. Graine aléatoire

L'option '`-s [N]`','`--seed [N]`' permet de spécifier la graine d'initialisation du générateur de nombres pseudo-aléatoires. Si cette option n'est pas précisée, le programme utilisera une graine aléatoire obtenue en mixant l'heure système et au moins un autre facteur aléatoire.

F9. Mélange des tuiles

Le programme doit implémenter deux algorithmes de mélange des tuiles : '`fisher-yates`' (l'algorithme de Fisher-Yates, aussi appelé Knuth shuffle) et '`naive`' (algorithme naïf qui échange chaque tuile avec une tuile aléatoire du paquet). L'option '`-x [SHUFFLE]`','`--shuffle [SHUFFLE]`' permet de spécifier l'algorithme utilisé. Par défaut, l'algorithme de Fisher-Yates sera utilisé.

Des tests statistiques doivent vérifier le comportement de chaque algorithme et comparer leur uniformité de distribution. Ces stratégies de tests devront être justifiées dans le rapport, ainsi que l'analyse des différences observées entre les deux algorithmes.

F10. Options spécifiques

L'option '`-p N`','`--players NUMBER`' permet de donner le nombre de joueurs (2 à 8). Par défaut, il y a 2 joueurs.

L'option '`-r`', '`--river`' permettra de jouer avec les tuiles de rivière en suivant les règles pour ces tuiles spéciales.

4 Gestion du jeu

Besoins communs aux deux interfaces (CLI et GUI).

F11. Gestion des règles

Le programme doit respecter les règles du jeu. Il doit être capable de gérer les coups joués par les joueurs, de vérifier leur validité et de mettre à jour l'état du jeu en conséquence. Il doit aussi pouvoir détecter la fin d'une partie, calculer les scores des joueurs et déterminer s'il y a un gagnant ou un match nul.

F12. Gestion des erreurs de l'utilisateur

Le programme doit être capable de gérer les erreurs de manière simple et compréhensive pour l'utilisateur. Si un utilisateur se trompe, le programme doit afficher un avertissement explicite et lui demander de recommencer.

F13. Gestion du *undo*/*redo*

Le programme doit afficher l'historique des coups joués et permettre le '*undo*' et le '*redo*'. Seuls les joueurs humains peuvent utiliser ces fonctionnalités. Lorsqu'un joueur humain fait un '*undo*', il annule son dernier coup joué et tous les coups joués par les joueurs artificiels précédents jusqu'à ce que l'on arrive à un autre joueur humain. Le '*redo*' permet de rejouer les coups annulés en suivant le même principe. Si le joueur humain joue un coup différent de celui qui a été annulé, l'historique futur est effacé et la partie continue normalement à partir de là.

F14. Gestion des conseils

Le programme doit être capable de donner des conseils à l'utilisateur sur le coup à jouer. Ces conseils doivent être basés sur l'état actuel du jeu et doivent être affichés à l'utilisateur sous forme compréhensible et simple.

5 Command Line Interface (CLI)

F15. Shell interactif

L'interface en ligne de commande doit se présenter sous la forme d'un shell interactif qui affiche un prompt à l'utilisateur et qui soit capable de lire les coups des joueurs ainsi que des commandes et d'y répondre. En cas d'erreur, le shell doit afficher un message d'erreur explicite.

```
>> COMMAND [OPTIONS] [ARGUMENTS]
```

Les commandes reconnues doivent être les suivantes :

- `new [ARGS]` : Lance une nouvelle partie, si des arguments sont ajoutés il est possible de jouer contre une IA, de choisir sa couleur, etc.
- `help [CMD]` : Affiche l'aide du shell ou de '`CMD`'.
- `quit` : Quitte le programme.
- `load FILE` : Charge une partie depuis un fichier.
- `save FILE` : Sauvegarde la partie en cours.
- `shuffle` : Mélange les cartes du jeu.
- `hint` : Affiche un conseil pour le coup à jouer.
- `undo [N]` : Annule le dernier coup joué (ou les `N`).
- `redo [N]` : Rejoue le dernier coup annulé (ou les `N`).
- `show hand` : Affiche la main actuelle du joueur.
- `show score` : Affiche le score actuel des joueurs.
- `show history` : Affiche l'historique des coups joués.
- `show configuration` : Affiche la configuration.
- `set PARAM=VALUE` : Change la configuration courante, par exemple : '`set debug=true`'.

F16. Proposer de sauvegarder avant de quitter

Si on quitte le programme qui héberge la partie et qu'aucune sauvegarde n'a été faite depuis le dernier coup joué, le programme doit demander si l'on veut sauvegarder la partie avant de quitter :

```
>> quit
Save the game before quitting? [y/N]
```

Si aucun choix reconnaissable n'est donné, le programme supposera que le choix par défaut a été choisi (`N`/*No*) et quitte le programme. Si on tape 'y' ou 'Y', le programme doit demander le chemin du fichier dans lequel sauvegarder et faire la sauvegarde. En cas d'erreur d'enregistrement, le programme doit redemander si l'utilisateur veut effectuer la sauvegarde.

F17. Édition de la commande en cours

Le shell doit permettre d'éditer la ligne de commande non encore validée en déplaçant le curseur avec les flèches pour permettre l'insertion ou la suppression de caractères. L'utilisation de bibliothèques comme GNU Readline (C), JLine (Java) ou readline (Python) est recommandée.

F18. Historique des commandes tapées

Le programme doit garder l'historique des commandes tapées par l'utilisateur. Il doit être possible de naviguer dans cet historique avec les flèches haut et bas du clavier. En tapant <Ctrl>+<R>, on doit pouvoir faire une recherche dans l'historique des commandes et obtenir la dernière commande qui correspond à la recherche.

F19. Complétion des commandes

Le programme doit être capable de compléter les commandes tapées par l'utilisateur. Par exemple, si l'utilisateur tape 'h' puis appuie sur la touche <Tab>, le programme doit afficher la commande qui correspond à 'h' si elle est unique, sinon un signal d'erreur doit être produit (clignotement du curseur ou son d'erreur) et un second appui sur la touche <Tab> affichera *toutes* les commandes commençant par 'h'.

F20. Notation des coups

Lorsqu'un joueur pose une tuile, il devra indiquer sur quelle tuile déjà posée il la colle, sur quelle face, avec quelle orientation et s'il y place un partisan. Par exemple, pour poser une tuile sur le côté nord de la tuile 3 du plateau en la tournant d'un quart de tour dans le sens horaire, on notera '`3N1`'. Si, en plus de cela on pose un partisan sur le côté est la tuile, il notera '`3N1/E`'. Il faut bien entendu faire précéder le coup de la tuile à poser en les séparant par un '`/`', ce qui donnera par exemple : '`RCFR/3N1/E`'. Le programme devra vérifier la validité du coup et signaler une erreur si le coup est invalide.

6 Format de sauvegarde

F21. Sauvegarde et restauration d'une partie

Les fichiers de sauvegarde doivent être au format texte ASCII. Le programme doit pouvoir convertir une partie en cours en un fichier à ce format et restaurer une partie à partir d'un fichier de sauvegarde. En cas d'erreur lors du chargement, afficher un message d'erreur explicite à l'utilisateur. Le parseur doit être robuste aux erreurs de format (par exemple, des lignes manquantes ou des valeurs incorrectes) et doit signaler les erreurs rencontrées.

La sauvegarde doit comporter les parties suivantes (dans l'ordre) : les paramètres du jeu (`settings`), l'état du jeu (`game`), et l'historique des coups joués (`history`).

F22. Commentaires dans la sauvegarde

La syntaxe de la sauvegarde doit gérer deux types de commentaires, les commentaires en ligne qui débutent par le caractère dièse ('`#`') jusqu'à la fin de la ligne, et

les commentaires en bloc délimités par les caractères accolades ('`{ }`').

F23. Format des paramètres du jeu

Les paramètres du programme sont, par exemple, le temps restant pour chaque joueur (en mode blitz), le nombre de joueurs de la partie, ... On les représente sous la forme d'une liste de couples clé-valeur comme suit :

```
[settings] # Paramètres du programme
debug=true
turns-limit=100
ai-mode=minimax
```

F24. Format du plateau de jeu

En mode ASCII *'à plat'*, les tuiles sont représentées symboliquement par la nature de leur quatre côtés. Les types de tuiles seront représentés par les lettres '`C`' (*City*), '`F`' (*Field*), '`R`' (*Road*) et, si l'extension *'The River'* est prise, '`S`' (*Stream* ou *River*). L'ordre des côtés sera toujours le même : nord, est, sud, ouest. Puis, si la tuile possède un élément spécial (monastère '`m`' ou bannière '`b`'), il sera indiqué par un caractère supplémentaire séparé par une barre oblique comme ceci : '`CFRS/m`'. Aussi, s'il y a une ambiguïté (par exemple une ville avec un champ), on utilisera une lettre minuscule pour indiquer ce qui occupe le centre de la tuile ('`c`' pour City, '`f`' pour Field, ...).

Par exemple, une tuile avec un chemin au nord '`R`', une ville à l'est '`C`', un champ au sud '`F`', un chemin à l'ouest et qui possède aussi une bannière sera représentée par '`RCFR/b`'.

Pour sauvegarder l'état du jeu, on utilisera une notation géographique en positionnant les tuiles sur un plan cartésien. Le centre de la tuile indique son numéro de placement avec, éventuellement, la présence d'un élément spécial (monastère '`m`' ou bannière '`b`').

La présence d'un pion (*Meeple*) sera indiquée par le nombre du joueur auquel il appartient placé sur une des faces de la tuile soit collé à la lettre représentant la face pour indiquer que le pion est au centre de la face, soit séparé par un slash pour indiquer qu'il est placé dans le cadrant immédiatement suivant le centre dans le sens horaire. Par exemple, '`R0`' indique que le joueur 0 a placé un pion sur la route alors que '`R/0`' indique qu'il est placé dans le cadrant suivant dans le sens horaire. Voici un exemple de graphe représentant un plateau de jeu :

```
[game]
1 # Joueur courant
# Graphe des tuiles
   R       C       F       F
 R[4]C -  C[2]R - R[0/m1]F - F[6/b]C
   R       R      R/1       F
   |       |       |
   R       R       R
F[5/2]R - R[3]C - C[1/c]C
   C       R       F
```

Dans cet exemple, on voit que le joueur 1 a placé un pion sur la route orientée vers le sud de la tuile 0 ainsi que sur le monastère de la même tuile. Le joueur 2 a placé un pion sur la route sud de la tuile 3 ainsi qu'un autre pion au centre de la tuile 5.

F25. Format de l'historique

L'historique sera représenté comme une liste des tours du jeu. Par exemple, le premier tour sera affiché comme suit : '`1:RCFR/3N1E; 2:FFFF/2S0;`' (le joueur puis la tuile qu'il pose et où il la pose).

```
[history]
0:FFFF;
1:FFCR/0W0; 2:CCRR/1W3/W;
...
```

7 Structures de données

F26. Module `graph`

On utilise un graphe sous la forme d'une liste d'adjacences (*adjacency list*) pour représenter l'état du plateau de jeu à chaque instant avec des algorithmes qui manipulent directement ces structures de données (plus court chemin, comptage de composantes connexes, ...). Ces structures de données et ces algorithmes seront rassemblés dans un module `graph` qui doit être aussi indépendant que possible du reste du programme.

F27. Algorithmes de manipulation des graphes

Des algorithmes de manipulation des graphes spécifiques au jeu doivent être développés, ils permettront au moins de :

- Vérifier si un coup est valide.
- Calculer les coups possibles pour un joueur.
- Appliquer un coup sur l'état du jeu.
- Calculer les scores des joueurs.
- Déterminer la fin de la partie.

8 Graphical User Interface (GUI)

F28. Menus de l'application

L'interface graphique doit comporter le menu '`File`' avec :

- `New Game` : Lancer une nouvelle partie.
- `Load Game` : Charger une partie depuis un fichier.
- `Save Game` : Sauvegarder la partie en cours.
- `Configuration` : Édition de la configuration.
- `Info` : Informations du logiciel (version, auteurs, ...).
- `Quit` : Quitter le programme.

Ainsi que le menu '`Game`' avec les items suivants :

- `Undo` : Annuler le dernier coup joué.
- `Redo` : Rejouer le dernier coup annulé.
- `Pause` : Arrêter le défilement du temps en blitz.
- `Hint` : Afficher un conseil pour le coup à jouer.

F29. Raccourcis clavier

Le GUI doit proposer les raccourcis clavier suivants :

- <Ctrl>+<N> : Nouvelle partie.
- <Ctrl>+<L> : Charger une partie sauvegardée.
- <Ctrl>+<S> : Sauvegarder la partie en cours.
- <Ctrl>+<,> : Afficher la configuration du logiciel.
- <Ctrl>+<I> : Afficher les informations du logiciel.
- <Ctrl>+<Q> : Quitter le programme.
- <Ctrl>+<U> : Annule le dernier coup joué.

- <Ctrl>+<R> : Rejoue le dernier coup annulé.
- <Ctrl>+<P> : Mettre le jeu en pause.
- <Ctrl>+<H> : Afficher un conseil de coup à jouer.

L'utilisateur doit pouvoir proposer ses propres raccourcis clavier pour chaque action listée. Les raccourcis clavier personnalisés seront sauvegardés dans le fichier de configuration du logiciel afin d'être restaurés à chaque lancement de l'application.

F30. *Drag'n'Drop*

Le *glisser-déposer* doit permettre de jouer à la souris en déplaçant les cartes sur la table de jeu.

9 Joueur artificiel

F31. Temps de réflexion borné

L'option `'--ai-time TIME'` permet de spécifier le temps de réflexion en secondes de l'algorithme de recherche. Si le temps est écoulé, le programme doit jouer le meilleur coup trouvé jusqu'à présent. Par défaut, le temps de réflexion sera de 5 secondes.

F32. Minimax avec $\alpha\beta$-pruning

Le programme doit comporter un algorithme de recherche de type minimax avec l'$\alpha\beta$-pruning pour évaluer les coups possibles. Cet algorithme est activé par défaut en mode joueur artificiel (option `'--ai-mode minimax'`).

F33. Fonctions d'évaluation pour Minimax

Au moins trois fonctions d'évaluation différentes doivent être implémentées pour évaluer les positions d'un état du jeu pour l'algorithme minimax. L'option `'--ai-minimax-scoring [SCORING]'` permet de spécifier la fonction d'évaluation choisie. Si aucune fonction n'est donnée, alors définissez une des fonctions comme le défaut.

F34. Approfondissement itératif de Minimax

L'option `'--ai-mode iterative'` remplace l'algorithme d'exploration par défaut par un algorithme d'exploration minimax avec approfondissement itératif (« *Iterative Deepening* »). L'approfondissement itératif permet de rechercher des coups de manière plus efficace en augmentant progressivement la profondeur de recherche en fonction du temps de réflexion disponible.

F35. Profondeur de recherche de Minimax

L'option `'--ai-minimax-depth DEPTH'` permet de spécifier la profondeur de recherche de l'algorithme de recherche. Si l'option `'--ai-minimax-depth DEPTH'` est donnée, `DEPTH` correspondra à la profondeur maximale de recherche. Sinon, la profondeur maximale sera déterminée automatiquement en fonction du temps disponible.

F36. Monte Carlo Tree Search (MCTS)

L'option `'--ai-mode mcts'` remplace l'algorithme d'exploration par défaut par un algorithme *Monte Carlo Tree Search* (MCTS) avec la fonction de sélection standard UCT (*Upper Confidence bounds applied to Trees*).

F37. Fonction de sélection par *Machine Learning*

Le programme doit proposer une fonction de sélection obtenue par apprentissage automatique. Utilisez un algorithme d'apprentissage simple (régression logistique, arbre de décision, etc.) entraîné sur une base de parties jouées. On ajoutera l'option `'--ai-mcts-selection SELECTION'` avec `SELECTION` à choisir parmi : `'UCT'` (défaut) et `'ML'`.

10 Jeu en réseau

F38. Découverte de serveurs et connexion client-serveur basique

Le programme doit implémenter les commandes réseau de base suivantes (en mode CLI mais aussi dans le GUI) :

- `server list` : Affiche la liste des serveurs de jeu disponibles sur le réseau local. Les serveurs diffusent périodiquement (toutes les 10 secondes) leur présence via un message UDP en broadcast sur le port 12346 contenant le nom du serveur et le port TCP. Un serveur est retiré de la liste si aucun message n'est reçu pendant 30 secondes.
- `server start [PORT]` : Démarre un serveur de jeu simple sur le port TCP indiqué (12345 par défaut). Le serveur doit pouvoir accepter une connexion client à la fois. Une fois démarré, le serveur diffuse sa présence sur le réseau via UDP broadcast. Si le port est déjà utilisé, afficher un message d'erreur.
- `server stop` : Arrête le serveur de jeu. Le client connecté est notifié de l'arrêt et déconnecté.
- `join [IP[:PORT]]` : Se connecte au serveur à l'adresse IP et au port indiqués (localhost :12345 par défaut). Une fois connecté, le programme passe en mode client. Si la connexion échoue, afficher un message d'erreur explicite.
- `quit` : Quitte le serveur et revient en mode local.
- `ping` : Envoie un message ping au serveur. Le serveur répond avec un pong et le temps de réponse est affiché. Cette commande permet de tester la connexion.

Le serveur doit gérer les déconnexions inopinées avec un timeout de 1 minute.

Les messages échangés entre client et serveur doivent être de simples mots en ASCII terminés par un retour chariot (`'\n'`). Par exemple :

```
Client: PING
Server: PONG TIME=42ms
Client: QUIT
Server: BYE
```

F39. Serveur multi-clients et gestion de parties

Le serveur doit être étendu pour gérer plusieurs clients simultanément en utilisant un thread par client ou de la programmation asynchrone. L'accès aux données partagées doit être protégé par des mécanismes de synchronisation (verrous/mutex) pour éviter les problèmes d'accès concurrent aux données.

Les commandes suivantes doivent être ajoutées :

- `server status` : Affiche le statut du serveur : port d'écoute, nombre de clients connectés, nombre de parties en cours.
- `players` : Affiche les joueurs connectés au serveur avec leur identifiant, nom, et statut (`idle`, `ingame`).
- `scoreboard` : Affiche le tableau des scores des joueurs qui ont joué sur le serveur (nombre de victoires, défaites, parties jouées).
- `new PLAYER_ID` : Démarre une nouvelle partie avec le joueur spécifié s'il est `idle`. Si le joueur n'existe pas ou n'est pas disponible, afficher une erreur. Si le

jeu accepte plus de deux joueurs, il suffit de mettre plusieurs joueurs en argument à la commande.

Le serveur doit maintenir l'état de toutes les parties en cours et router les commandes de jeu (coups) vers la bonne partie. Chaque coup joué par un client est validé par le serveur puis transmis à l'autre joueur.

Le programme doit aussi proposer les options de lancement suivantes :

- `'-s [PORT]'` ou `'--server [PORT]'` : Lance directement en mode serveur sur le port indiqué.
- `'-d'` ou `'--daemon'` : Lance le serveur en mode headless (sans interface).

Exemple de protocole pour les coups :

```
Client1: MOVE e2-e4
Server -> Client2: OPPONENT_MOVE e2-e4
Client2: MOVE e7-e5
Server -> Client1: OPPONENT_MOVE e7-e5
```

Le serveur doit valider que les coups sont légaux pour empêcher la triche (coups invalides, actions hors tour).

F40. Système d'invitations et gestion avancée des joueurs

Le système de création de parties doit être amélioré avec un mécanisme d'invitation et d'acceptation :

- `players [PLAYER_ID]` : Si un ID est fourni, affiche les informations détaillées du joueur (statut, score, statistiques). Les statuts possibles sont maintenant : `idle`, `away`, `waitgame` (en attente de réponse à une invitation), `ingame`.
- `new PLAYER_ID` : Envoie une invitation au joueur spécifié. Le joueur invité reçoit une notification et peut accepter (`accept`) ou refuser (`decline`) l'invitation. Si aucune réponse n'est reçue après 5 minutes, l'invitation expire. Le demandeur peut annuler l'invitation avec `cancel`.
- `accept` : Accepte l'invitation en cours. La partie démarre immédiatement.
- `decline` : Refuse l'invitation. L'inviteur est notifié.
- `cancel` : Annule l'invitation envoyée (si en attente).
- `away` : Change le statut du joueur en `away`. Les invitations ne peuvent être envoyées aux joueurs `away`.
- `back` : Revient au statut `idle`.

Le serveur doit gérer les timeout d'invitation (5 minutes), notifier les joueurs des changements d'état, et empêcher les actions invalides (inviter un joueur déjà en partie, accepter sans invitation, etc.). Exemple d'interaction :

```
Client1: NEW 2
Server -> Client1: INVITATION_SENT
                   PLAYER=Bob TIMEOUT=300s
Server -> Client2: INVITATION_RECEIVED
                   FROM=Alice EXPIRES=300s
Client2: ACCEPT
Server -> Client1: INVITATION_ACCEPTED
                   STARTING_GAME
Server -> Client2: GAME_START
                   OPPONENT=Alice
```

A Les Tuiles utilisées

Les tuiles utilisées dans le jeu de base sont listées ci-dessous (empruntées à la page Wikipedia du jeu [4]).

A.1 Tuiles de base

Tuile	Encodage	Nombre
	FFFF/m	×4
	FFFR/m	×2
	CFFF	×5
	FCFC/f	×3
	CFCF/c	×1
	CFCF/b	×2
	CCCC/b	×1
	FFCC	×2

Tuile	Encodage	Nombre
	CCFF	×3
	CCFF/b	×2
	CCCF	×3
	CCCF/b	×1
	CCCR	×1
	CCCR/b	×2
	FRFR	×8
	FFRR	×9
	CRFR	×4
	CFRR	×3
	CRRF	×3
	CCRR	×3
	CCRR/b	×2
	FRRR	×4
	CRRR	×3
	RRRR	×1

A.2 Tuiles de l'extension *'The River'*

Tuile	Encodage	Nombre
	FSRS/m	×1
	FFSF	×2
	FSFS	×2
	FFSS	×2
	CSCS	×1
	CCSS	×1
	CSRS	×1
	RSRS	×1
	RRSS	×1

Références

[1] Fred Valdez Ameneyro, Edgar Galván, and Ángel Fernando Kuri Morales. Playing Carcassonne with monte carlo tree search. *Computing Research Repository (CoRR)*, abs/2009.12974, 2020.

[2] Thomas Hauk, Michael Buro, and Jonathan Schaeffer. Rediscovering *-Minimax Search. 3846 :35–50, 2004.

[3] Cathleen Heyden. Implementing a computer player for Carcassonne. Master Thesis, June 2009.

[4] Wikipedia. Carcassonne (board game). `https://en.wikipedia.org/wiki/Carcassonne_(board_game)`, July 2025. Accessed : 2025-07-17.